QueueLess Government Office

Hackathon Problem Statement — PS-2

> **A system that predicts waiting time at government offices and allows citizens to take a virtual token remotely, arriving only when their turn is approaching.**

---

1. Core Idea

QueueLess Government Office is a web-based queue and token management system for government offices.

The main purpose is to:

- Allow citizens to take a **virtual token online**.
- Let citizens know their **expected waiting time**.
- Reduce unnecessary physical waiting at government offices.
- Allow citizens to arrive **only when their turn is approaching**.
- Help government offices manage **multiple counters/windows** efficiently.
- Handle both **online and offline citizens**.
- Provide administrators and office staff with a real-time view of the queue.

Main Concept

Citizen → Select Government Service → Get Virtual Token → Track Queue → Receive Live Notification → Arrive at Office → QR Check-in → Service

---

2. Government Services

The initial examples mentioned in the notes are:

- Birth Certificate
- Driving License
- Other government services

The system should be designed so that **multiple services can be added later**.

---

3. Major Features

3.1 Online Virtual Token

Citizens should be able to:

- Select the required government service.
- Take a token remotely.
- See their token number.
- See the estimated waiting time.
- Know approximately when they need to reach the office.

The citizen should **not have to physically stand in the queue just to obtain a token**.

3.2 Waiting-Time Prediction

This is one of the most important parts of the project.

The system should calculate/predict:

- Current queue length
- Number of people ahead
- Average service time
- Number of active counters/windows
- Current token being served
- Estimated waiting time
- Peak-hour conditions

The objective is to give the citizen something like:

> **Your Token: A-24**
> **People Ahead: 7**
> **Estimated Waiting Time: 42 minutes**
> **Recommended Arrival: 10:35 AM**

---

4. Token & Time Logic

The notes suggest that the system should use **time-based slots**.

Initial concept

- Each token/service may be allocated approximately **20–30 minutes**.
- The exact duration can depend on the service.
- The system should dynamically calculate waiting time rather than simply assigning a fixed appointment time.

Examples from the notes:

- 20–25 minutes
- 20–30 minutes
- 15–30 minutes

These appear to be rough estimates for service/token duration and should be treated as **configurable values**, not hard-coded values.

---

5. Government Office Working Hours

The handwritten notes specify:

Office Timing

9:00 AM – 5:00 PM

Break

1:00 PM – 2:00 PM

The token system therefore needs to understand:

- Opening time
- Closing time
- Break period
- Available service windows
- Remaining capacity for the day

Tokens should **not be scheduled during the official break period**.

---

6. Daily Token Capacity

The notes mention:

> **20 tokens per day**

This appears to be an initial proposed limit.

So the system can potentially support:

- Maximum daily token capacity
- Remaining tokens
- Tokens already issued
- Available slots
- Offline token allocation

Example:

> **Today's Capacity: 20 tokens**
> **Tokens Issued: 13**
> **Remaining: 7**

The exact number should ideally be configurable by the administrator.

---

7. Online + Offline Token System

An important point from the notes is that **not every citizen will necessarily use the online system**.

Therefore, the system needs to support:

Online Citizens

Citizen gets a token through the web application.

Offline Citizens

Government-office staff can issue a token manually to citizens who arrive physically.

This means the same queue should contain:

- Online tokens
- Offline tokens

Both should be handled by the same queue-management system.

---

8. Live Queue Updates

The queue should update in real time.

Citizens should be able to see:

- Current token being served
- Their token
- Number of people ahead
- Estimated waiting time
- Counter/window assigned
- Queue status

For example:

> **Now Serving: B-17**
> **Your Token: B-24**
> **6 people ahead**
> **Estimated wait: 35 minutes**

---

9. Live Notifications

The notes specifically mention **live notifications**.

Notifications can be triggered when:

- Your token is approaching.
- Your token is called.
- Your counter/window is assigned.
- You are expected to reach the office.
- You have missed your turn.
- The queue is delayed.
- The estimated waiting time changes significantly.

This is what allows the project to achieve its main goal:

> **Citizens don't need to sit in a government-office queue for hours.**

---

10. QR Check-In

The notes mention **QR check-in**.

The idea can work like this:

Before arriving

Citizen receives a virtual token.

At the office

Citizen scans a QR code.

System

The system verifies:

- Token
- Citizen
- Service
- Arrival/check-in status

This can help prevent fake or duplicate token usage.

---

11. Multiple Counters / Windows

Government offices may have multiple service counters.

Therefore, the system should support:

- Counter 1
- Counter 2
- Counter 3
- etc.

Different counters can potentially handle different services.

The queue-management system should know:

> Which counter is available?
>
> Which counter is serving which token?
>
> Which counter is currently busy?
>
> Which counter will become available next?

---

12. Peak-Hour Handling

The notes specifically discuss **peak hours** and changing physical waiting time.

This is important because the same token duration cannot always be assumed.

During peak hours:

- Queue size increases.
- Waiting time increases.
- More counters may need to be activated.
- Estimated waiting time should be recalculated.

Therefore:

Normal period

Lower queue → shorter predicted waiting time.

Peak period

Higher queue → longer predicted waiting time.

The system should dynamically adapt rather than showing a fixed waiting time.

---

13. AI Component

The notes mention **AI** and **LLM training**.

The AI portion can be used primarily for **prediction and intelligent queue management**.

Possible AI functionality:

Waiting-Time Prediction

Input:

- Current queue
- Historical service time
- Number of counters
- Current token
- Service type
- Time of day
- Peak-hour data
- Office break timings

Output:

Predicted waiting time.

Future Enhancement

The system could learn from historical data.

For example:

Monday 10 AM
Average service time = 17 min

Monday 2 PM
Average service time = 11 min

Friday 11 AM
Average service time = 24 min

Over time, the system becomes better at predicting waiting times.

---

14. Multilingual Support

The notes explicitly mention:

Multiple language

This is particularly important for a government-office application.

The application should potentially support:

- English
- Gujarati
- Hindi
- Other regional languages

The exact languages can be decided based on the target users.

The goal is to make the application accessible to citizens who may not be comfortable using English.

---

15. Citizen Dashboard

The notes contain a rough dashboard sketch.

The dashboard should contain important information such as:

Token

- Current token
- Upcoming token
- Token status
- People ahead

Calendar

A calendar/appointment-style view can show:

- Selected date
- Token date
- Expected service time

Other possible information

- Service selected
- Estimated waiting time
- Counter/window
- Check-in status

---

16. User / Citizen Module

The notes divide the system into different user roles.

Citizen/User

The citizen-side functionality includes:

Dashboard

- Token status
- Upcoming token
- Waiting time
- Queue status

List of Documents

Citizen can see/check the documents required for the selected service.

For example:

Driving License

- Required document 1
- Required document 2
- Required document 3

Document Verification

The system/staff can verify whether required documents are available.

Token Agent / Token Management

Citizen can:

- Generate token
- View token
- Track token
- Manage token status

Out Time / Expected Arrival

The notes mention **"out time"**.

This can be interpreted as the expected time at which the citizen should leave/reach based on their queue position.

---

17. Approval / Staff Module

The second role in the notes appears to be an **Approval/Staff** role.

Responsibilities include:

Area Availability

Check:

- Available service area
- Available counters
- Available capacity

Token Management

Staff can see:

- Tokens waiting
- Tokens currently being served
- Completed tokens

Document Verification

Staff can verify citizen documents before/while providing the service.

Complete Token

Once service is finished:

Token → Completed

This allows the system to accurately calculate the remaining queue.

---

18. Admin Module

The Admin has higher-level control.

The notes mention:

Area Management

Admin can manage:

- Government-office areas
- Counters/windows
- Service areas

Approval List

Admin can view/manage:

- Approval requests
- Verification
- Staff/service approvals

Verification

Admin can verify or manage the approval status of relevant requests.

---

19. Basic Role Structure

The overall system can therefore be organized as:

```
            QUEUELESS GOVERNMENT OFFICE
                        |
        ------------------------------------
        |               |                |
     CITIZEN          STAFF            ADMIN
        |               |                |
   Dashboard       Token Management   Area Management
   Documents       Queue Management   Approval List
   Token           Document Verify    Verification
   Waiting Time    Complete Token
   Notifications   Counter Management
   QR Check-in
```

---

20. Suggested End-to-End Workflow

Step 1 — Citizen opens application

↓

Step 2 — Selects government service

Example:

Birth Certificate

↓

Step 3 — System shows required documents

↓

Step 4 — Citizen checks available capacity

System shows:

Tokens Available: 7/20

↓

Step 5 — Citizen requests virtual token

↓

Step 6 — System generates token

Example:

> **Token: B-14**

↓

Step 7 — AI/system calculates estimated waiting time

Example:

> **Estimated waiting: 55 minutes**

↓

Step 8 — Citizen does not need to wait physically

Citizen can continue their normal activities.

↓

Step 9 — Live queue updates

Example:

Current Token: B-09
Your Token: B-14

People Ahead: 4

Estimated Waiting Time:
~32 minutes

↓

Step 10 — Notification

When the token approaches:

> **Your turn is approaching. Please reach the government office.**

↓

Step 11 — Citizen reaches office

↓

Step 12 — QR Check-in

↓

Step 13 — Counter/window assigned

Example:

Proceed to Counter 3

↓

Step 14 — Service is provided

↓

Step 15 — Token marked complete

↓

Step 16 — Queue automatically updates

---

21. Queue Calculation Concept

The rough notes contain several calculations involving:

- 30 minutes
- 10 minutes
- 3 minutes
- 15 minutes
- 5 minutes
- 20–25 minutes
- 20–30 minutes
- 15–30 minutes
- 7 hours
- 30 minutes

These appear to be rough calculations for determining **how many tokens can realistically be handled within office hours and how waiting time changes based on service duration**.

A cleaner system should therefore calculate:

Available Working Time
÷
Average Service Time

=

Approximate Daily Capacity

But this should also account for:

- 1–2 PM break
- Multiple counters
- Different service durations
- Peak hours
- Offline tokens
- Delays
- Completed tokens

So the actual system should use **dynamic queue calculation**, rather than simply:

Number of tokens × fixed time.

---

22. Technology Stack Mentioned in Notes

The handwritten notes mention:

Backend

Python

Possible responsibility:

- API
- Queue logic
- Waiting-time prediction
- AI/ML
- Business logic

Database

MongoDB

Potential data:

- Users
- Tokens
- Services
- Counters
- Documents

- Queue history
- Office timings
- Notifications
- Historical waiting-time data

Frontend

The notes mention:

- React.js
- Node.js
- Angular.js

However, **React.js and Angular.js are both frontend frameworks**, while Node.js is generally used for backend/server-side JavaScript.

For the actual implementation, the team should decide on **one frontend framework** rather than trying to use both React and Angular.

For example:

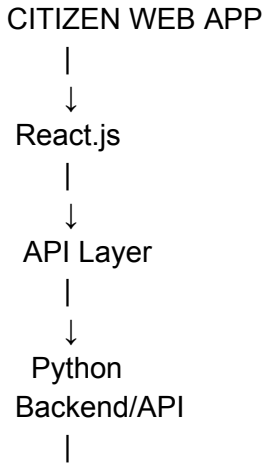
Frontend → React.js
Backend → Python
Database → MongoDB
AI/ML → Python

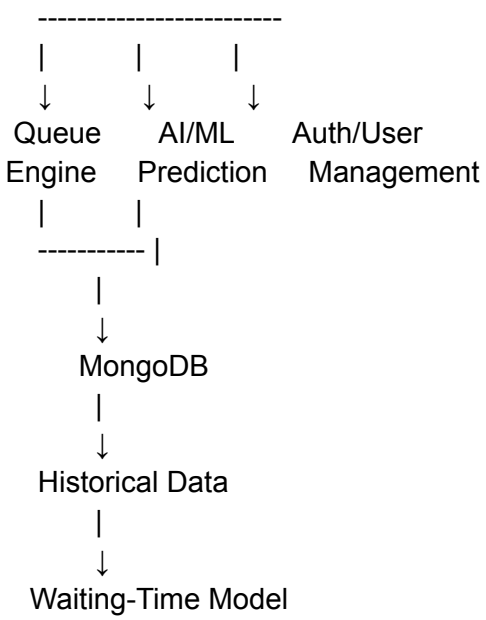
Node.js can be used if there is a specific reason for it, such as a Node-based real-time notification/service layer.

---

23. Proposed Technical Architecture

A clean architecture based on the notes could be:





---

24. Important Database Entities

Based on everything in the notes, the system will likely need collections/tables for:

Users

user_id
name
mobile/email
language

Services

service_id
service_name
required_documents
average_service_time

Tokens

token_id
token_number
user_id
service_id
date
status
estimated_waiting_time
counter
created_at

Counters

counter_id
counter_name
available
current_token
service_type

Queue History

token
service_time
waiting_time
counter
date
time

This historical data can later help the AI prediction system.

---

25. Dashboard Structure

A cleaner dashboard based on the handwritten UI sketch:

```
-----------------------------------------------
|            QUEUELESS                        |
|            Dashboard                        |
-----------------------------------------------
|                                             |
|    CURRENT TOKEN       CALENDAR             |
|         B-24              OCT 7             |
|                                             |
|    People Ahead: 6                          |
|    Estimated Wait: 35 min                   |
|                                             |
-----------------------------------------------
|            QUEUE STATUS                     |
|                                             |
|    Now Serving: B-18                        |
|    Your Token: B-24                         |
|    Counter: 2                               |
|                                             |
-----------------------------------------------
|      Documents       Notifications          |
-----------------------------------------------
```

---

26. Navigation Bar

The notes explicitly mention a **Navigation Bar**.

A possible structure:

Home
Dashboard
Take Token
My Tokens
Documents
Notifications
Profile

For Admin/Staff:

Dashboard
Queue
Tokens
Counters
Services
Approvals
Reports
Settings

---

27. Security / Token Validation

The system should prevent problems such as:

- Duplicate tokens
- Fake tokens
- Reusing completed tokens
- Someone taking multiple tokens unnecessarily
- Missing check-in
- Unauthorized staff access

Possible mechanisms:

- Login/authentication
- Unique token ID
- QR code
- Token expiry
- Role-based access control
- Admin/staff authentication

---

28. Missed Token Handling

Since citizens are not physically waiting in the queue, there needs to be a mechanism for **missed turns**.

For example:

Token B-24 called
↓
Citizen doesn't check in
↓
Grace period
↓
Token marked "Missed"
↓
Queue moves forward

The exact grace period can be decided by the team.

---

29. Key Innovation of the Project

The strongest part of the project isn't simply:

> "Online token booking."

The actual innovation should be presented as:

> **Predictive virtual queuing — allowing citizens to stay away from the government office until their estimated service time approaches.**

Traditional system:

Citizen
↓
Government Office
↓
Take Token
↓
Wait
↓
Wait
↓
Wait
↓
Service

QueueLess:

Citizen
↓
Online Token
↓
AI Predicts Waiting Time
↓
Citizen Goes About Their Work
↓
Live Notification
↓
Arrives Near Turn
↓
QR Check-in
↓
Counter
↓
Service

---

30. The Problem We Are Actually Solving

The project addresses:

- Long physical queues
- Unpredictable waiting times
- Crowded government offices
- Wasted citizen time
- Poor visibility of queue status
- Manual queue management
- Lack of real-time updates
- Difficulty handling peak hours
- Inefficient counter utilization
- Citizens arriving too early
- Citizens missing their turn
- Lack of accessibility for different languages

---

31. Final Feature Set

For the hackathon, the complete feature list from the notes can be consolidated into:

Citizen

- Online token
- Government-service selection
- Required document list
- Document verification/status
- Virtual queue
- Waiting-time prediction
- Live queue tracking
- Live notifications
- QR check-in
- Token status
- Expected arrival time
- Multiple-language support
- Dashboard
- Calendar
- Offline-token support through staff

Staff

- Staff dashboard
- Current queue
- Available area/counter
- Token management
- Document verification
- Call next token
- Complete token
- Counter management
- Offline token creation
- Queue status

Admin

- Area management
- Counter/window management
- Service management
- Approval list
- Verification
- Token/capacity management
- Staff management
- Queue monitoring

AI/System

- Waiting-time prediction
- Peak-hour detection
- Dynamic queue calculation
- Historical-data analysis
- Counter utilization
- Estimated arrival time

- Real-time recalculation

---

32. Important Numbers / Constraints From Mentor Notes

Keep these as **initial project assumptions to validate with the mentor**, because they were written as rough calculations:

Item	Initial Note
Government office	9 AM – 5 PM
Break	1 PM – 2 PM
Daily token capacity	20 tokens/day
Approx. service/token duration	20–30 min
Other rough durations	15–30 min / 20–25 min
Token type	Online + Offline
Queue updates	Live
Check-in	QR
Counters	Multiple
Languages	Multiple
AI	Waiting-time prediction
Database	MongoDB
Backend	Python
Frontend mentioned	React.js / Angular.js
Other technology mentioned	Node.js
AI/LLM	LLM / training mentioned

---

33. One Important Clarification From the Notes

There are a few handwritten items whose exact wording isn't clear enough to safely convert into requirements:

- **"AI attack" / "AI attract"** — likely an AI-related feature, but exact meaning is unclear.
- Some **10/10, 20/10, 15/10** calculations on the role/queue page.
- Some of the **30 min / 10 min / 3 min / 15 min / 5 min** calculations.
- The exact meaning of **"Token agent"**.
- The exact meaning of **"Out time"**.
- A few labels around **"Area Available Token / pre-list / doc token / complete token."**

I would **not delete these from the project**, but I also wouldn't turn my guesses into official requirements. These should be confirmed with your mentor/sir before implementation.

---

34. Clean One-Line Project Definition

If you need to explain the entire project quickly to a judge/mentor:

> **QueueLess Government Office is an AI-powered virtual queue management system that allows citizens to take government-service tokens remotely, predicts their waiting time using real-time and historical queue data, and notifies them when they should arrive—reducing physical queues, overcrowding, and wasted time.**

This gives you a much cleaner foundation for turning the rough notebook notes into the **actual system architecture, database schema, UI screens, API flow, and hackathon implementation plan**.